Business Problem Statement: Distribution-Based Risk Clustering using Wasserstein Distance

Financial institutions and analytics teams rely heavily on clustering techniques to group assets, portfolios, or customers based on similar characteristics. Traditional clustering approaches, such as K-Means, typically use Euclidean distance, which assumes point-wise similarity. However, in financial contexts, data is often better represented as distributions (e.g., return distributions, risk profiles, volatility patterns) rather than single-point estimates.

As a result, conventional clustering methods may fail to capture the true underlying similarities between assets, especially when distributions differ in shape, variance, or tail behavior. This limitation can lead to suboptimal grouping, inaccurate risk segmentation, and ineffective portfolio decisions.

A risk analytics team aims to develop an advanced clustering framework that **leverages distribution-aware distance metrics**, such as Wasserstein Distance (Earth Mover's Distance), to more accurately group financial assets based on their return distributions and risk characteristics.

The goal is to build an intelligent analytics system that can compare traditional clustering approaches with distribution-based clustering, generate meaningful risk segments, and provide interpretable insights for decision-making.

You are tasked with developing an end-to-end analytics solution to address the following business question:

"How can distribution-based distance metrics like Wasserstein Distance be leveraged to improve clustering of financial assets and generate more accurate and interpretable risk segmentation compared to traditional methods?"

Deliverables

1. *Data Engineering & Modeling (Python)*
    - Generate synthetic distribution-based datasets (or use sampled distributions)
    - Represent each data point as a small distribution (vector sample)
    - Implement preprocessing utilities for normalization and transformation
    - Implement baseline clustering using Euclidean K-Means
    - Develop a custom Wasserstein K-Means algorithm from scratch
    - Assign clusters based on distribution similarity
    - Evaluate clustering stability and separation visually and statistically
2. *Data Storage & Querying (Local File System / CSV)*
    - Store generated synthetic datasets in structured CSV format
    - Save clustering results (cluster labels per sample)
    - Store intermediate outputs for reproducibility
    - Enable loading and reuse of datasets in notebooks and API
3. *Visualization & Insights (Plotly / Interactive Python Visualization)*
    - Develop interactive visualizations for cluster comparison

- Plot histograms of distributions grouped by clusters
- Compare cluster assignments across iterations
- Visualize cluster separation and similarity
- Enable interactive exploration of clustered distributions

4. API Development (FastAPI)

- Develop RESTful APIs to serve clustering outputs
- Endpoints for:
    - Running Wasserstein K-Means clustering
    - Returning cluster labels for input distributions
    - Health check endpoint for service validation
- Enable integration between ML core logic and external applications
- Implement clean schema validation using Pydantic
- Ensure modular service-based architecture

5. AI & Intelligence Layer

- Core focus on Wasserstein distance-based similarity computation
- Custom clustering logic for distributional data
- Optional analytical enhancements:
    - Simple cluster interpretation based on distribution patterns
    - Basic statistical summaries per cluster (mean-like behavior of distributions)
- Generate interpretable grouping of distribution samples for analysis